

Archon Specs: A Deterministic Architecture Compiler and Continuous Contract Reconciliation Control Plane for AI-Assisted Software Engineering

Hernan Humana¹ et al.
¹Independent Systems Researcher
contact@archonspecs.org

Abstract

Large Language Models (LLMs) have demonstrated remarkable capabilities in localized program synthesis; however, their direct deployment for end-to-end enterprise software engineering is fundamentally compromised by two critical failure modes: *semantic drift* and *token explosion*. Free-form code synthesis delegates architectural orchestration, schema enforcement, dependency wiring, and infrastructural boilerplate entirely to probabilistic auto-regressive generation. This paradigm incurs an $\mathcal{O}(|AST| \times T_{\text{boilerplate}})$ token overhead per iteration, frequently hallucinates incompatible relational foreign keys or imports, and progressively violates bounded domain contexts as codebases scale.

To resolve these pathology classes, we present **Archon Specs**, an open-source, deterministic architectural compiler and continuous contract reconciliation control plane. Archon Specs decomposes software synthesis into a closed-loop governance cycle: high-level intent is elicited into formally validated UML schemas and sharded JSON contracts (Requirement, Architecture, and Domain Shards), compiled into a normalized Intermediate Representation (DesignSpec IR), and materialized into production-grade systems via a deterministic, zero-LLM Abstract Syntax Tree (AST) synthesis pipeline. Real-time developer and multi-agent coordination is mediated through a unified Model Context Protocol (MCP) server exposing over 30 architectural primitives alongside an Electron-based observability workstation. We formalize the contract space, the AST masking operator, the cryptographic fingerprinting function, and the non-destructive 3-way AST merge lattice. Empirical evaluation across synthetic and real-world microservice and modular monolith benchmarks demonstrates that Archon Specs reduces LLM token consumption by **86.3%–89.0%**, enforces **100% adherence** to declared architectural invariants with zero compilation errors, and achieves drift scan throughput exceeding **26,000 artifacts/second**. Archon Specs is publicly available under the MIT license at <https://github.com/herhu/archon-specs>.

1 Introduction

The integration of Large Language Models (LLMs) into software development workflows has inaugurated a fundamental transition from manual implementation to prompt-guided program synthesis [1–4]. Modern autonomous coding agents [5] iteratively inspect file trees, issue patch commands, and attempt to resolve feature requests or defects. Despite these operational advances, an acute structural gap persists between localized code emission and large-scale architectural governance.

When applied to multi-module, distributed, or enterprise software stacks, free-form LLM code generation exhibits two compounding pathology classes:

1. **Semantic and Architectural Drift:** LLMs operate without an authoritative global invariants graph. As multi-turn agentic conversations evolve, auto-regressive sampling inevitably violates cross-domain encapsulation boundaries, invents phantom type signatures, generates conflicting database foreign keys, and fractures inversion-of-control (IoC) dependency graphs. This represents an acute manifestation of classical software architecture erosion [6–8], accelerated by probabilistic hallucination.
2. **Token Inefficiency and Context Explosion:** Enterprise applications are overwhelmingly dominated by structural and infrastructural boilerplate: dependency injection wiring, Data Transfer Object (DTO) validation decorators, database schemas and migrations, OpenAPI/Swagger annotations, and distributed tracing instrumentation. In unconstrained agentic frameworks, the model is forced to emit every token of this boilerplate redundantly. When modifying a single domain field, the agent re-synthesizes hundreds of lines of code, resulting in an asymptotic token complexity of $\mathcal{O}(|AST| \times T_{\text{boilerplate}})$. This consumes prohibitive context window budget, induces multi-agent amnesia, and increases operational costs.

To overcome these structural limitations, we introduce **Archon Specs**, a deterministic architecture compiler and continuous contract reconciliation control plane for

AI-assisted software engineering. The core premise of Archon Specs is a strict *separation of concerns*: LLMs should be employed exclusively as cognitive translators that map ambiguous business requirements to formal architectural contracts, whereas the physical realization of the system must be governed by a deterministic, non-probabilistic AST compiler.

Archon Specs introduces a continuous reconciliation control plane operating over formal contract shards. Intent is captured through visual or textual Unified Modeling Language (UML) specifications (supporting ASCII DSL, PlantUML, and Mermaid grammars) and partitioned into independently governed JSON Schema contracts. An Intermediate Representation (IR) compiler reconciles and hardens these shards into a canonicalized DesignSpec, computing a content-addressed cryptographic fingerprint. A deterministic AST materializer then emits a production-grade modular monolith architecture based on NestJS, complete with domain-driven design (DDD) boundaries, an in-process CQRS event bus, role-based access control (RBAC), and automated DDL migrations.

To govern interaction between human architects and autonomous agent swarms (e.g., Claude, Cursor, Anti-gravity), Archon Specs implements a 30+ tool Model Context Protocol (MCP) server [9] coupled with an Electron/Vite desktop workstation for real-time visual inspection, shard locking, and diff auditing.

This paper presents the following contributions:

- **Architecture & Control Plane:** We specify the Archon Specs architecture, detailing the multi-agent MCP tool bus and the Electron-based developer control workstation.
- **Formal Reconciliation Loop:** We formalize the continuous contract reconciliation loop, including contract sharding, invariant validation predicates, cryptographic spec canonicalization, and a non-destructive 3-way AST merge operator supporting @ArchonManual regions.
- **Deterministic AST Materializer:** We describe the compilation of normalized specifications into production-grade NestJS modular monoliths and provide a formal complexity proof establishing an $\mathcal{O}(|C|)$ token bound versus $\mathcal{O}(|AST| \times T_{\text{boilerplate}})$ for free-form synthesis.
- **Empirical Evaluation:** We evaluate Archon Specs across three scaling tiers (Small, Medium, Enterprise). Results demonstrate an **86.3%–89.0% token reduction**, **100% adherence** to declared architectural invariants, and millisecond-level compilation and drift detection scaling.
- **Open Source Implementation:** We release Archon Specs as a fully tested, MIT-licensed open-

source ecosystem at <https://github.com/herhu/archon-specs>.

2 Background & Motivation

2.1 Architectural Erosion in Agentic Workflows

Software architecture research has long established that without strict formal enforcement, systems inevitably suffer from architectural drift and decay [6, 7, 10]. Classical Architecture Description Languages (ADLs) such as Wright [11], Darwin [12], and Acme [13] provided mathematical rigor for component-connector configurations but struggled with industry adoption due to tooling friction and divergence between abstract specifications and realized code.

In modern agentic workflows, autonomous LLM agents synthesize code by modifying raw source files directly. Because the model’s effective context is constrained, it cannot maintain the global topological graph of large codebases. This manifests in: (i) circular module dependencies; (ii) leaking of persistence primitives across service boundaries; (iii) orphaned database relations; and (iv) disparate error-handling conventions. In empirical evaluations, over 40% of multi-file patches generated by unconstrained agents introduce latent structural violations or syntax incompatibilities [4, 5].

2.2 The Model-Driven Architecture (MDA) Promise

Model-Driven Architecture (MDA) [14–16] historically proposed deriving executable artifacts from Platform-Independent Models (PIMs). However, classical MDA failed to bridge the “semantic execution gap”: hand-coded business logic had to be painstakingly woven into rigid code skeletons, and re-running generators would obliterate human edits.

Archon Specs revives and transforms the MDA philosophy for the AI era. By leveraging modern AST parsing tools (**ts-morph**) and formalizing a semantic masking function μ , Archon Specs enables *Strict Similitude*: models and code remain bi-directionally synchronized without destroying developer-crafted logic.

2.3 Model Context Protocol (MCP) as an Architectural Bus

Recently, the Model Context Protocol (MCP) [9] has emerged as an open standard for connecting AI models to external tools and context providers. Archon Specs utilizes MCP not merely as a passive retrieval utility, but as a *stateful architectural control plane* that governs workspace scaffolding, schema validation, shard locking, diff simulation, and atomic file materialization.

3 Archon Architecture & Control Plane

Figure 1 illustrates the multi-tier topology of Archon Specs. The system is architected around three cooperative planes: the *Interaction Plane*, the *Continuous Contract Reconciliation Core*, and the *Target System Plane*.

3.1 The Interaction Plane: Workstation & Agents

The interaction plane bridges human system architects and autonomous AI agents:

- **Desktop Workstation (GUI):** Implemented in Electron and Vite, the Archon Workstation serves as mission control. It provides deep architectural observability: interactive UML component and sequence graph visualizations, real-time shard inspection, structural diff previews, and distributed trace telemetry. Architects can manually lock domain shards, inspect pending compiler mutations, and enforce human-in-the-loop approvals.
- **Autonomous AI Agents:** Tool-augmented agents (including Claude Desktop, Cursor, and Antigravity swarms) interact with Archon through standard MCP RPC protocols. Agents operate within defined persona boundaries (e.g., Lead Architect, Shard Visualizer, Platform Builder) without requiring direct write access to disk during architectural synthesis.

3.2 The Unified 30+ Tool MCP Control Plane

The Model Context Protocol (MCP) server functions as the universal bidirectional bus. Rather than exposing arbitrary shell execution or raw file modification primitives, the Archon MCP server restricts agent capabilities to strongly typed architectural operations. Table 1 summarizes representative primitives across the five core tool categories.

3.3 Virtual File System (VFS) and Stream Materialization

To guarantee transactional safety, Archon decouples code generation from physical disk I/O. The compiler generates changes into an in-memory *Virtual File System (VFS)*. Mutations are scheduled as an ordered sequence of atomic *ChangeOperations* (CREATE, UPDATE, PATCH).

During execution, the CLI acts as a physical layer client listening to a Server-Sent Events (SSE) stream emitted by the MCP engine (PLAN_STREAM_STARTED → PLAN_OPERATION_READY → PLAN_STREAM_COMMITTED).

If network communication drops or an invariant fails mid-stream, the VFS transaction is rolled back, preventing partial or corrupted project states.

4 Contract Sharding & Reconciliation

The heart of Archon Specs is the *Continuous Contract Reconciliation Loop*, which continuously synchronizes abstract intent, intermediate representations, and concrete source code.

4.1 Formal Contract Space

Definition 1 (Contract Space). *A system specification S is defined as a tuple:*

$$S = \langle \mathcal{R}, \mathcal{A}, \mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_k, \mathcal{C} \rangle \quad (1)$$

where:

- $\mathcal{R}$ is the **Requirement Contract**, governing functional business rules $\mathcal{B}$, system actors $\mathcal{X}$, and invariant predicates $\mathcal{I}$.
- $\mathcal{A}$ is the **Architecture Contract**, defining bounded contexts, target database engines, and cross-cutting infrastructural configurations.
- $\mathcal{D}_i = \langle K_i, \mathcal{E}_i, \mathcal{S}_i \rangle$ is the i -th **Domain Shard**, where K_i is a unique domain key, $\mathcal{E}_i$ is the set of domain entities with typed attributes and foreign key references, and $\mathcal{S}_i$ is the set of service operations.
- $\mathcal{C}$ is the **Cross-Cutting Capability Shard**, defining shared authorization schemes, pagination standards, and distributed tracing policies.

Each contract shard is strictly validated against draft-07 JSON Schema specifications:

$$\forall \mathcal{D}_i, \quad \mathcal{D}_i \models \Sigma_{\text{Domain}}, \quad \mathcal{R} \models \Sigma_{\text{Req}}, \quad \mathcal{A} \models \Sigma_{\text{Arch}} \quad (2)$$

4.2 Shard Sharding and Distributed Governance

In multi-agent collaborative workflows, concurrent mutations to a single monolithic specification file trigger race conditions and semantic collisions. Archon Specs enforces *Contract Sharding*: each bounded domain (e.g., identity, billing, inventory) is isolated into an independent file shard.

Agents must acquire a shard lock via `archon_lock_shard(domainKey)` before emitting modifications. The control plane enforces lease expirations and validates that cross-domain foreign key references $\text{FK}(e_a \in \mathcal{D}_i \rightarrow e_b \in \mathcal{D}_j)$ reference entities that are publicly exported by bounded context $\mathcal{D}_j$.

Table 1: **Representative MCP Tool Suite in Archon Specs.** Unified architectural control plane exposed to human architects and autonomous agent swarms.

Category	Tool Name	Architectural Function
UML Engine	<code>uml_parse_ascii</code>	Compiles raw ASCII/PlantUML DSL into DiagramIR.
	<code>uml_validate_ir</code>	Validates topological soundness of components.
	<code>uml_ir_to_designspec</code>	Maps conceptual diagrams to typed JSON schemas.
	<code>archon_generate_diagram</code>	Synthesizes synchronized Mermaid markdown.
Governance	<code>archon_lock_shard</code>	Acquires distributed mutex on a domain shard.
	<code>archon_release_shard</code>	Releases shard lock with hash validation.
	<code>archon_diff_spec</code>	Computes structural AST deltas between revisions.
	<code>archon_validate_shard</code>	Evaluates JSON Schema and domain boundary rules.
Compilation	<code>archon_compile_shards</code>	Merges distributed shards into global DesignSpec.
	<code>archon_fingerprint</code>	Computes SHA-256 canonical content address.
	<code>archon_plan_project</code>	Generates deterministic change execution plan.
Materializer	<code>archon_workflow_start</code>	Initiates governed execution state machine.
	<code>archon_generate</code>	Invokes template engine on Virtual File System.
	<code>archon_reconcile</code>	Scans disk, masks manual code, logs drift.

4.3 Cryptographic Fingerprinting & Strict Similitude

To detect architectural divergence instantaneously, Archon Specs introduces a canonical content-addressed fingerprinting function $h(S)$.

Definition 2 (Canonicalization Operator). *Let κ be an operator that transforms an arbitrary specification S into an invariant normal form $\kappa(S)$ by recursively sorting:*

1. Domain shards lexicographically by unique key K_i ;
2. Entities within each domain alphabetically by entity name;
3. Entity fields alphabetically by field name, with primary keys hoisted;
4. Relationships and foreign keys by `name ◦ targetEntity`;
5. Service operations alphabetically by method identifier.

The cryptographic fingerprint $h(S)$ is computed as:

$$h(S) = \text{Trunc}_{12}(\text{SHA-256}(\text{StableSerialize}(\kappa(S)))) \quad (3)$$

Because the UML visualization engine and the code generator share the exact canonicalizer κ , Archon guarantees *Strict Similitude*: if and only if the generated Mermaid class diagram and the TypeScript entity codebase share the exact hash $h(S)$, documentation and implementation are mathematically proven to be identical.

4.4 AST Masking and 3-Way Reconciliation

A critical requirement in real-world software engineering is preserving human-written business logic across repeated compiler regenerations. Archon Specs solves this

through semantic AST masking and surgical 3-way AST merging.

Definition 3 (Semantic Masking Operator μ). *Let $\mathcal{T}$ be a TypeScript Abstract Syntax Tree. The masking operator $\mu : \mathcal{T} \rightarrow \mathcal{T}_{\text{masked}}$ traverses $\mathcal{T}$ and replaces all subtrees tagged with either:*

1. A comment delimiter: `//@archon-manual-start:id` ... `//@archon-manual-end;` or
2. A decorator node: `@ArchonManual()` preceding a method or class declaration,

with an invariant null placeholder token ϵ_{mask} .

Let $\mathcal{H}(c)$ denote the SHA-256 hash of source code string c . When the reconciliation engine scans an existing disk artifact $f \in \mathcal{F}$, it evaluates:

$$h_{\text{full}}(f) = \mathcal{H}(\text{content}(f)), \quad h_{\text{masked}}(f) = \mathcal{H}(\mu(\text{content}(f))) \quad (4)$$

The drift classification function $\Delta(f)$ is formalized as:

$$\Delta(f) = \begin{cases} \text{MISSING}, & \text{if } f \notin \text{Disk} \\ \text{NODRIFT}, & \text{if } h_{\text{full}}(f) = h_{\text{rec}}(f) \\ \text{EVOLUTION}, & \text{if } h_{\text{full}}(f) \neq h_{\text{rec}}(f) \wedge \\ & h_{\text{masked}}(f) = h_{\text{rec-mask}}(f) \\ \text{UNSAFEDRIFT}, & \text{if } h_{\text{masked}}(f) \neq h_{\text{rec-mask}}(f) \end{cases} \quad (5)$$

where $h_{\text{rec}}(f)$ and $h_{\text{rec-mask}}(f)$ denote the recorded baseline hashes.

When EVOLUTION is detected, the developer has modified business logic inside protected manual zones, leaving compiler-managed boilerplate untouched. The 3-way AST merger then executes non-destructive reconciliation:

- **Imports:** New module imports required by schema updates are additively injected; existing imports are preserved.
- **Class Properties:** Added entity columns are inserted with appropriate TypeORM decorators; existing properties are retained.
- **Constructors:** Dependency injection parameters are reconciled additively by matching injected token symbols.
- **Methods:** Unprotected methods are re-materialized to reflect new service signatures, while `@ArchonManual()` methods are extracted from the old AST and surgically spliced into the newly generated AST without modification.

5 Deterministic AST Materializer & Target System

5.1 Target Architecture: Modular Monolith

Rather than generating fragmented microservices that suffer from distributed network latency, cascading failures, and complex orchestration overhead [17–19], Archon Specs compiles into a high-performance **Modular Monolith** architecture based on NestJS and TypeScript [20].

The generated system enforces strict domain boundaries:

1. **Domain Modules & CQRS:** Each domain shard compiles into an isolated NestJS module containing encapsulated TypeORM entities, domain services, DTOs with `class-validator` pipes, and controller endpoints. Cross-domain interactions are strictly mediated via in-process CQRS event buses or explicit injectable interface contracts.
2. **Security & Governance:** Authentication is provisioned with JWT and OIDC strategies. Role-Based Access Control (RBAC) is enforced through declarative NestJS guards and interceptors.
3. **Infrastructure & Observability:** The compiler automatically synthesizes PostgreSQL/SQLite DDL auto-migrations, self-documenting OpenAPI 3.0 (Swagger) specifications, containerized multi-stage Docker builds, and W3C distributed tracing context propagation.

5.2 Mathematical Token Complexity Analysis

We now provide a formal analysis comparing the token complexity of Archon Specs against unconstrained free-form LLM code generation.

Definition 4 (System Sizes). *Let a target software system contain N entities distributed across M bounded domains. Let F denote the average number of fields per entity, and O denote the average number of service operations per domain.*

Theorem 1 (Token Complexity Bound). *Free-form LLM synthesis incurs token complexity:*

$$\begin{aligned} T_{\text{free-form}} &= \mathcal{O}(|\mathcal{AST}| \times K) \\ &= \mathcal{O}((N \cdot F + M \cdot O) \cdot C_{\text{boilerplate}} \cdot K) \end{aligned} \quad (6)$$

where $C_{\text{boilerplate}} \gg 1$ is the expansion factor of concrete source boilerplate, and $K \geq 1$ is the number of interactive repair prompts required to resolve syntax and drift errors.

In contrast, Archon Specs achieves token complexity:

$$T_{\text{Archon}} = \mathcal{O}(|\mathcal{C}|) = \mathcal{O}(N \cdot F + M \cdot O) \quad (7)$$

independent of $C_{\text{boilerplate}}$ and with materialization token cost identically zero:

$$T_{\text{materialize}} = 0 \text{ LLM tokens} \quad (8)$$

Proof. In free-form synthesis, the generative model must explicitly emit the lexical stream of all target source files. For each entity e , the model produces: (i) an entity definition file with ORM decorators; (ii) create, update, and response DTO files with validation decorators; (iii) repository wrappers; (iv) service business skeletons; (v) controller routes; and (vi) module wiring. Each semantic field of length $\mathcal{O}(1)$ in the abstract model expands to $C_{\text{boilerplate}} \approx 40\text{--}80$ tokens of concrete TypeScript code across these layers. Because LLMs exhibit non-zero error rates on complex multi-file relationships, K prompt-response iterations are required to resolve missing imports or compilation breaks, yielding $T_{\text{free-form}} = \mathcal{O}(|\mathcal{AST}| \cdot K)$.

In Archon Specs, the LLM only generates or modifies the compact contract shard $\mathcal{D}_i$ or the high-level UML DSL. The length of this contract depends strictly on entity and field names: $|\mathcal{C}| = \sum_{i=1}^M (|K_i| + \sum_{e \in \mathcal{E}_i} |\text{fields}(e)|) = \mathcal{O}(N \cdot F + M \cdot O)$.

The transformation from $\mathcal{C}$ to concrete source code $\mathcal{F}$ is executed entirely by the deterministic AST compilation engine $\mathcal{M} : \mathcal{C} \rightarrow \mathcal{F}$ implemented in native TypeScript and Handlebars. Because $\mathcal{M}$ is executed by deterministic CPU instructions without invoking language model inference:

$$\text{Cost}_{\text{LLM}}(\mathcal{M}) = 0 \quad (9)$$

Hence, total LLM token consumption is strictly bounded by $\mathcal{O}(|\mathcal{C}|)$. $\square$

6 Empirical Evaluation

We conducted extensive empirical evaluations to measure: (1) token consumption reduction; (2) architectural

Tier	Entities	Free-Form Tokens	Archon Tokens	Reduction
Small	5	42,500	5,820	86.3%
Medium	50	312,000	34,250	89.0%
Enterprise	200	1,240,000	142,600	88.5%

Table 2: **LLM Token Consumption and Reduction Rates.** Comparison of generative tokens consumed during system materialization.

drift prevention; (3) spec compilation latency; (4) drift scan throughput; and (5) worker queue scalability.

6.1 Experimental Setup

Benchmarks were executed on an Apple M-series workstation running macOS Darwin 24.6.0 with Node.js v20.19.0. We evaluated three architectural tiers:

- **Tier 1 (Small):** 1 domain, 5 entities (~ 5 entities total; equivalent to a localized microservice).
- **Tier 2 (Medium):** 5 domains, 10 entities per domain (~ 50 entities total; standard multi-module backend).
- **Tier 3 (Enterprise):** 10 domains, 20 entities per domain (~ 200 entities total; large-scale modular monolith).

6.2 Token Consumption and Cost Reduction

Table 2 compares LLM token consumption between baseline free-form generation (using Claude 3.5 Sonnet to emit code files iteratively) and Archon Specs contract synthesis.

Across all tiers, Archon Specs achieves an average token reduction of **87.9%**. The reduction is most pronounced in enterprise tiers because repeated infrastructural scaffolding (TypeORM decorators, Swagger metadata, CRUD handlers) is handled completely by the deterministic materializer at zero LLM token cost.

6.3 Architectural Drift Prevention

To evaluate drift resistance, we generated 500 feature evolutions across the benchmark tiers, injecting foreign key additions, entity renaming, and custom business logic edits inside `@ArchonManual` blocks.

- **Free-Form Baseline:** Suffered an architectural drift failure rate of **38.4%**. Common failures included broken module imports in NestJS (`Cannot find module`), mismatched relational types between SQLite and PostgreSQL, and silent deletion of previously written manual methods.
- **Archon Specs:** Maintained a **100% contract adherence rate** (0 drift violations). In all 500 runs,

Tier	Entities	Mean Latency	Min / Max (ms)	Throughput
Small	5	1.2 ms	0.8 / 2.1	4,166 ent/s
Medium	50	4.6 ms	3.9 / 6.2	10,869 ent/s
Enterprise	200	18.7 ms	16.2 / 23.4	10,695 ent/s

Table 3: **Spec Compilation and Normalization Latency.** In-memory IR compilation and SHA-256 fingerprinting.

Managed Artifacts	Scan Duration	Throughput
100	3.8 ms	26,315 art/s
1,000	37.9 ms	26,385 art/s
10,000	361.4 ms	27,670 art/s

Table 4: **Drift Detection Scaling.** Linear $\mathcal{O}(N)$ scanning performance across managed files on disk.

TypeScript type-checking (`tsc -noEmit`) and ESLint validation executed with 100% clean passes. 100% of manual regions protected with `@ArchonManual()` were preserved intact through 3-way AST merging.

6.4 Compiler & Materialization Throughput

As shown in Table 3, full specification compilation and canonical fingerprinting for a 200-entity enterprise system requires only 18.7 milliseconds, demonstrating throughput exceeding 10,000 entities per second.

Table 4 details drift scanner scalability. When monitoring a repository containing 10,000 managed source artifacts with synthetic drift injected across 10% of files, Archon’s dual-hash masking algorithm completed the full scan in **361.4 ms**, maintaining a constant throughput of over 27,000 artifacts per second.

6.5 Worker Pool Concurrency Simulation

We benchmarked the MCP worker pool dispatcher across concurrency levels $C \in \{1, 5, 10, 20\}$ processing batches of 200 architectural operations. At concurrency $C = 20$, the pool achieved a throughput of **1,428 operations/second** with an average queue wait time of **0.48 ms** ($p95 < 1.9$ ms), verifying that the MCP control plane introduces negligible scheduling overhead.

7 Related Work

LLM-Based Coding Agents: Systems such as SWE-agent, Devin, and Cursor have popularized tool-augmented coding agents [4, 5]. However, these tools operate primarily at the level of individual source files, lacking formal invariant validation or architectural models. Archon Specs provides an orthogonal control plane

that can sit above these agents, confining them to validated contract modifications.

Software Architecture Description Languages (ADLs): Classical ADLs such as Wright [11], Darwin [12], and Acme [13] provided mathematical rigor for component models. Modern frameworks such as SysML and PlantUML offer graphical descriptions but lack continuous AST reconciliation. Archon Specs operationalizes ADL concepts by integrating directly into contemporary TypeScript compilers and JSON Schema validators.

Model-Driven Architecture (MDA) and Generative Programming: Generative techniques [14–16] demonstrated the value of deriving code from high-level models. Archon Specs advances this field by introducing non-destructive AST merging, semantic masking (μ), and cryptographic content addressing, overcoming the historic friction of generator round-tripping.

8 Conclusion & Availability

We have presented **Archon Specs**, a deterministic architecture compiler and continuous contract reconciliation control plane designed to eliminate semantic drift and token explosion in AI-assisted software engineering. By constraining generative models to validated contract shards and executing physical materialization through a deterministic AST compiler, Archon Specs achieves an **87.9% average token reduction** while guaranteeing **100% adherence** to architectural invariants.

Archon Specs is fully open-sourced under the MIT license. Source code, documentation, benchmarks, and installation packages are publicly available at:

<https://github.com/herhu/archon-specs>

References

- [1] Y. Li, D. Choi, J. Chung, N. Kushman, J. Schrittwieser, R. Leblond, T. Eccles, J. Keeling, F. Gimeno, A. Dal Lago *et al.*, “Competition-level code generation with AlphaCode,” *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
- [2] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, T. Remez, J. Rapin *et al.*, “Code llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2023.
- [3] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, “Large language models for software engineering: A systematic literature review,” *ACM Computing Surveys*, 2023.
- [4] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, and J. Grundy, “Large language models for software engineering: Survey and outlook,” *ACM Transactions on Software Engineering and Methodology*, 2024.
- [5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.
- [6] D. E. Perry and A. L. Wolf, “Foundations for the study of software architecture,” *ACM SIGSOFT Software Engineering Notes*, vol. 17, no. 4, pp. 40–52, 1992.
- [7] J. Rosik, J. Le Gear, J. Buckley, and M. A. Babar, “An industrial case study of architectural drift in commercial software systems,” *Empirical Software Engineering*, vol. 16, no. 2, pp. 204–245, 2011.
- [8] D. M. Le, D. Link, A. Shahbazian, and N. Medvidovic, “An empirical study of architectural decay in open-source software,” in *Proceedings of the 37th International Conference on Software Engineering (ICSE)*, 2015, pp. 176–186.
- [9] Anthropic, “Model context protocol (MCP) specification,” <https://modelcontextprotocol.io/>, 2024.
- [10] N. Medvidovic and R. N. Taylor, “A classification and comparison framework for software architecture description languages,” *IEEE Transactions on Software Engineering*, vol. 26, no. 1, pp. 70–93, 2000.
- [11] R. Allen and D. Garlan, “A formal basis for architectural connection,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 6, no. 3, pp. 213–249, 1997.
- [12] J. Magee and J. Kramer, “Dynamic structure in software architectures,” in *Proceedings of the 4th ACM SIGSOFT Symposium on Foundations of Software Engineering (FSE)*, 1996, pp. 3–14.
- [13] D. Garlan, R. T. Monroe, and D. Wile, “Acme: Architectural description of component-based systems,” *Foundations of Component-Based Systems*, pp. 47–68, 2000.
- [14] A. Kleppe, J. Warmer, and W. Bast, *MDA Explained: The Model Driven Architecture: Practice and Promise*. Addison-Wesley Longman Publishing Co., Inc., 2003.
- [15] K. Czarnecki and U. W. Eisenecker, *Generative Programming: Methods, Tools, and Applications*. Addison-Wesley, 2000.
- [16] D. Steinberg, F. Budinsky, E. Merks, and M. Paterostro, *EMF: Eclipse Modeling Framework*, 2nd ed. Addison-Wesley Professional, 2008.

- [17] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.
- [18] G. Blinowski, A. Ojdana, and A. Przybyłek, "Monolithic vs. microservice architecture: A performance and scalability evaluation," *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022.
- [19] M. Fowler, "Monolithfirst," *martinfowler.com*, 2015. [Online]. Available: <https://martinfowler.com/bliki/MonolithFirst.html>
- [20] G. Bierman, M. Abadi, and M. Torgersen, "Understanding typescript," in *Proceedings of the 28th European Conference on Object-Oriented Programming (ECOOP)*. Springer, 2014, pp. 257–281.

